Hands-On-Learning: Build and Validate a Product Search System

Description

This activity guides you through building a complete embedding-based semantic search system for an e-commerce product catalog. You'll work with a realistic scenario where keyword-based search is failing customers, and your task is to implement a solution using modern NLP techniques. By generating semantic embeddings with sentence-transformers, constructing efficient FAISS vector indices, and applying quantitative validation through recall metrics, you'll develop practical skills in search system development and quality assessment that are essential for production ML engineering.

Learning Objectives

By the end of this activity, learners will be able to:

- Generate semantic embeddings from product descriptions using sentence-transformers.
- Build efficient FAISS vector indices for similarity search across thousands of products.

- Calculate recall@5 and recall@10 metrics using test query sets with ground truth labels.
- Analyze failure cases to identify specific query patterns or categories with poor performance.

Assignment Components

Introductory Slide

Title: ML Engineer Mission: Fix TechMart's Broken Search Before Q4

Subtitle: Transform 47% search satisfaction to 60%+ using embeddings, FAISS, and recall validation

Tools used:

- Python 3.9+ with pandas and NumPy
- Sentence-transformers library (all-MiniLM-L6-v2 model)
- FAISS library for vector search
- Jupyter Notebook or Python IDE
- CSV file with product catalog data

Scenario

You are a machine learning engineer at TechMart, a mid-sized e-commerce company specializing in consumer electronics. TechMart's current keyword-based search system is producing poor results—only 47% of users find what they're looking for on the first page, leading to cart abandonment and lost revenue. Your manager has tasked you with prototyping an embedding-based semantic search system to improve product discovery within two weeks.

TechMart sells 5,000 electronics products across categories like laptops, phones, headphones, cameras, and tablets. Customer support receives 200+ complaints weekly about search relevance—users searching for "wireless earbuds" get wired headphones, searches for "laptop stand" return keyboards. The business goal is to increase search satisfaction from 47% to at least 60% in the next quarter, measured by recall@5 and recall@10 metrics on a representative test query set.

You have access to a CSV file with 5,000 products containing product_id, title, description, category, and price fields. Some products have incomplete descriptions or missing metadata. You'll use Python with sentence-transformers to generate embeddings, FAISS to build a search index, and a curated test query set to validate performance. Success means achieving at least 55% recall@5 (an 8+ percentage point improvement over the 47% baseline) and identifying specific failure patterns to guide future improvements.

Instructions for Learners

Step 1: Data Preparation and Cleaning

Download the TechMart product catalog CSV from the provided repository.

Load the data using pandas and inspect the first 10 rows to understand the schema.

Check for missing values in title and description columns—these are critical for embeddings.

Create a combined text field by concatenating title and description: combined_text = title + " " + description.

Handle missing descriptions by filling with empty string or using title only.

Remove any duplicate products based on product_id.

Step 2: Generate Embeddings with Sentence-Transformers

Install sentence-transformers: pip install sentence-transformers.

Import the library and load the all-MiniLM-L6-v2 model.

Generate embeddings for all products using model.encode() with batch_size=32 for efficiency.

Verify the embedding shape: should be (5000, 384) for 5000 products with 384 dimensions.

Save embeddings to disk as a NumPy array: np.save('product_embeddings.npy', embeddings).

Step 3: Build FAISS Index for Vector Search

Install FAISS: pip install faiss-cpu.

Create a Flat L2 index: index = faiss.IndexFlatL2(384).

Add product embeddings to the index: index.add(embeddings).

Verify the index size matches your product count: index.ntotal should equal 5000.

Test a sample search: encode a query like "wireless headphones", search the index, and examine top-5 results.

Step 4: Validate Recall with Test Query Set

Download or create a test query set with 50 queries covering all product categories.

For each query, define ground truth: a list of relevant product_ids (typically 3-10 per query).

For each test query: (a) encode the query text, (b) search the FAISS index for top-10 results, (c) extract product_ids of retrieved results.

Calculate recall@5: (number of relevant products in top-5) / (total relevant products).

Calculate recall@10: (number of relevant products in top-10) /
(total relevant products).

Average recall@5 and recall@10 across all 50 queries to get overall metrics

Step 5: Analyze Failure Cases

Identify queries with recall@5 < 40% (significantly below target).

Group failures by product category—do certain categories (e.g., cameras, tablets) perform worse?

Examine 5-10 specific failure cases: what products should have been retrieved but weren't?

Document patterns: Are failures due to synonym mismatches ("laptop" vs "notebook"), multi-word queries, products with poor descriptions, or model limitations?

Write a brief summary (3-5 sentences) describing the top 3 failure patterns you identified.

Compile Your Findings

Requirement 1: Complete Python Implementation

A fully functional Python implementation of the TechMart product search system.

Include: Data loading and cleaning code, embedding generation using sentence-transformers, FAISS index creation, search functionality, and recall calculation logic with clear comments explaining each component.

Requirement 2: Recall Metrics Report

A comprehensive report of your search system's performance metrics.

Include: Overall recall@5 and recall@10 scores, category-level breakdowns for laptops, phones, headphones, cameras, and tablets, and comparison against the 47% baseline and 55% target thresholds.

Requirement 3: Failure Analysis Document

A detailed analysis of queries where your search system underperformed.

Include: Top 3 failure patterns identified, specific query examples demonstrating each pattern, root cause analysis (synonym mismatches, poor descriptions, model limitations), and proposed solutions for each failure type.

Requirement 4: Production Readiness Assessment

A reflection on whether your system meets deployment criteria and lessons learned.

Include: Evaluation against the 55% recall@5 target, key insights about validating search quality, limitations discovered during testing, and recommended next steps for production deployment.

Summary

This activity provides hands-on experience with semantic search system development through practical application of embedding generation, vector indexing, and quantitative validation techniques. Participants have developed competency in using sentence-transformers for NLP embeddings, building FAISS indices for efficient similarity search, and calculating recall metrics for quality assessment, which are directly applicable to production search systems in e-commerce and enterprise applications.

Share The Project

Document: Save the report in PDF format.

Upload: Share in the “Peer Review” area of the course shell with a brief description.

Instructions for Documenting and Sharing The Project for Peer Review

1. Document the Project

- Use word processing software to create your report.
- Ensure all sections of the project document structure are completed.
- Proofread and edit your report for clarity and accuracy.

2. Share the Project

- Save your report in PDF format.
- Upload your documents to the “Peer Review” area in the course shell.
- Provide a brief description of your project in the submission post.

Instructions for Documenting and Sharing The Project for Peer Review

3. Peer Review

- Review the projects submitted by your peers.
- Provide constructive feedback and comments on their work.